The Problem

Lizards need to cross a driveway to find food, but too many lizards crossing simultaneously will alert cats and the cats will “play” with them. In programming terms, lizard threads will attempt to cross the driveway for a certain number of seconds, eat for a random number of seconds, then attempt to cross back and sleep. The two cat threads will sleep for a certain amount of time then wake up and check the amount of lizard threads crossing the driveway. If that number exceeds the set number of maximum lizards crossing allowed, the program terminates. The overall problem to solve is to make sure all lizards cross safely within the set amount of time given. The code solves this problem by only allowing a lizard to cross the driveway when it is safe to do so and ensuring not too many lizards cross at the same time.

Changes to Code

Discussion of Code Changes

At the global scope, a POSIX semaphore named lizard_semaphore was declared and initialized to MAX_LIZARD_CROSSING (4) to cap driveway concurrency, alongside a std::mutex named crossing_mutex to protect shared global state from data races. For the cat threads, the Cat::run(), Cat::wait(), and catThread loop functions were completed to allow cat threads to periodically sleep and safely inspect the active driveway tracking counters.

Within the lizard crossing and safety functions, sem_wait(&lizard_semaphore) was integrated inside sago2MonkeyGrassIsSafe() and monkeyGrass2SagoIsSafe() to throttle driveway access efficiently without relying on busy-wait loops. Shared counter increments and decrements for both directions were wrapped within crossing_mutex.lock() and crossing_mutex.unlock() blocks to eliminate race conditions. Additionally, sem_post(&lizard_semaphore) was added inside madeIt2MonkeyGrass() and madeIt2Sago() to signal precisely when a lizard has safely exited the pavement, freeing up a crossing slot for waiting threads.

Finally, the main() function was fully implemented to initialize semaphores and the random seed, instantiate dynamic thread vectors for both lizards and cats, and execute .run() to launch concurrent execution. It manages the simulation timer using WORLDEND, signals a graceful shutdown by setting running = 0, joins all active threads via .wait(), and thoroughly cleans up all heap memory and semaphores using delete and sem_destroy.

Results

WORLDEND (s)	Max Num of Lizards Crossing	Lizards safe?
30	4	Yes

120	4	Yes
240	4	Yes

Issues Encountered

The main challenge during development was figuring out whether a mutex or a semaphore was the right tool for each part of the simulation—using semaphores to control driveway capacity while using a mutex to protect the shared tracking counters. Another critical issue discovered was handling potential early exits: if the simulation encountered an error condition and called exit(-1) while holding the mutex lock, it would cause a deadlock or leave shared data corrupted. This was resolved by ensuring the mutex is always properly unlocked before any termination or exit call is executed, allowing the system to safely release shared resources.